

Chapter 1 Notes

Unit I: Introduction to Web Technology

[3 Hrs.]

1. Web Basics

1.1 Internet, Intranet, WWW

Internet - Global network of interconnected computer networks. - Uses standard protocol suite **TCP/IP** to communicate. - Public, open to everyone worldwide. - Example: browsing any public website, sending email.

Intranet - Private network, restricted to an organization. - Built using same technologies as Internet (TCP/IP, HTTP) but access limited to employees/members. - Protected by firewall from outside access. - Example: a company's internal HR portal, internal wiki.

WWW (World Wide Web) - System of interlinked hypertext documents accessed via Internet. - Invented by **Tim Berners-Lee** in 1989. - Uses **HTTP/HTTPS** protocol, **URLs** to locate resources, **HTML** to structure pages. - Web is a *service* that runs on top of Internet — Internet is the infrastructure, WWW is one application using it (others: email, FTP).

Feature	Internet	Intranet	WWW
Access	Public	Private/restricted	Public (part of Internet)
Scope	Global network of networks	Organization-only network	Collection of web pages/documents
Protocol	TCP/IP	TCP/IP	HTTP/HTTPS

1.2 Static and Dynamic Web Pages

Static Web Page - Content fixed, same for every user, every visit. - Written in plain HTML/CSS, stored as-is on server. - Server sends file to browser without processing. - Fast, simple, no database needed. - Example: a basic company "About Us" page that never changes.

Dynamic Web Page - Content generated/changes based on user input, time, database, or other conditions. - Requires server-side processing (or client-side scripting) to build page before/after sending. - Example: Facebook feed, online shopping cart, search results page.

Static	Dynamic
Content same for all users	Content varies per user/request
No server processing needed	Needs server-side script/database
Faster to load	Slightly slower (processing involved)
Easy to build	Needs backend language (PHP, Node.js, Python, etc.)

1.3 Web Clients

- Software (usually browser) that **requests** resources from a web server.
- Examples: Chrome, Firefox, Safari, Edge, or any app making HTTP requests (mobile app, curl).
- Responsibilities:
 - Send HTTP request to server (with URL).
 - Receive HTTP response.
 - Render HTML/CSS, execute JavaScript, display page to user.

1.4 Web Servers

- Software/hardware that **stores, processes, and delivers** web pages to clients.
- Listens for incoming HTTP requests, sends back HTTP responses (HTML, JSON, files, etc.).
- Examples: Apache HTTP Server, Nginx, Microsoft IIS, Node.js (with Express).
- Responsibilities:
 - Accept client request.
 - Locate/generate requested resource.
 - Send response back (status code + data).

Request-Response cycle (client-server basic flow):

```
Client (browser) --- HTTP Request ---> Web Server
Client (browser) <--- HTTP Response --- Web Server
```

2. Client-Server Architecture

General model: clients send requests, servers process and reply. Differ by number of **tiers/layers** separating presentation, logic, and data.

2.1 Single Tier (1-Tier)

- All three layers — presentation, business logic, data — reside on **one machine**.
- No network involved for the app to function.
- Example: a desktop application storing data in local file (e.g., simple standalone software, MS Access on one PC).
- Pros: simple, fast (no network latency).
- Cons: not scalable, no multi-user support, hard to maintain centrally.

2.2 Two-Tier

- **Client** (presentation + some logic) directly talks to **Server** (database/data layer).
- Example: desktop app connecting directly to a database server (client-side app + DB server).
- Pros: simple, decent performance.
- Cons: business logic often mixed into client, harder to update logic (must update every client), limited scalability, security risk (client has direct DB access).

2.3 Multi-Tier (N-Tier, typically 3-Tier)

- Layers split across **three or more** independent tiers:
 1. **Presentation Tier** — UI, browser/client.
 2. **Application/Logic Tier** — business logic, application server.
 3. **Data Tier** — database server.
- Example: typical modern web app — browser → web/app server (Node.js/Django/PHP) → database (MySQL/MongoDB).
- Pros: scalable, maintainable, secure (client never touches DB directly), logic centralized/reusable.
- Cons: more complex to design/deploy, more network hops (latency).

```
Single-Tier:  [ UI + Logic + Data ]   (one machine)
```

```
Two-Tier:    [ Client (UI+Logic) ] <---> [ Database Server ]
```

```
Multi-Tier:  [ Client/UI ] <---> [ App Server (Logic) ] <---> [ Database Server ]
```

3. HTTP: HTTP Request and Response

HTTP (HyperText Transfer Protocol) — application-layer protocol for transferring web content, is **stateless** (each request independent, no memory of previous request).

HTTP Request — structure

1. **Request Line** — Method + URL + HTTP version `GET /index.html HTTP/1.1`
2. **Headers** — metadata (Host, User-Agent, Accept, Content-Type, Cookie...)
3. **Body** (optional) — data sent (used in POST/PUT)

Common HTTP Methods: | Method | Purpose | |—|—| | GET | Retrieve data/resource | | POST | Submit data to server (create) | | PUT | Update existing resource (full) | | PATCH | Update existing resource (partial) | | DELETE | Remove resource |

HTTP Response — structure

1. **Status Line** — HTTP version + Status Code + Reason phrase `HTTP/1.1 200 OK`
2. **Headers** — metadata (Content-Type, Content-Length, Set-Cookie...)
3. **Body** — actual content (HTML, JSON, image, etc.)

Common Status Codes: | Code | Class | Meaning | |—|—|—| | 200 | 2xx Success | OK | | 301 | 3xx Redirection | Moved Permanently | | 400 | 4xx Client Error | Bad Request | | 401 | 4xx Client Error | Unauthorized | | 404 | 4xx Client Error | Not Found | | 500 | 5xx Server Error | Internal Server Error |

Full cycle example:

```
Browser → GET /home.html HTTP/1.1
        Host: www.example.com

Server  → HTTP/1.1 200 OK
        Content-Type: text/html
        <html>...page content...</html>
```

4. URL (Uniform Resource Locator)

Address used to locate a resource on the web.

Structure:

```
scheme://host:port/path?query#fragment

https://www.example.com:443/products/list?id=5&sort=asc#reviews
```

Part	Example	Meaning
Scheme/Protocol	https://	Protocol to use (http, https, ftp)
Host/Domain	www.example.com	Server address
Port	:443	Port number (optional, default 80/443)
Path	/products/list	Location of resource on server
Query String	?id=5&sort=asc	Extra parameters sent to server
Fragment	#reviews	Section within the page (client-side only)

5. Client-Side Scripting

- Code that **runs in the browser** (on user's machine), after page delivered.
- Language: **JavaScript** (primary), also older: VBScript.
- Used for: form validation, DOM manipulation, animations, interactivity, AJAX calls — without reloading page.
- Pros: fast response (no server round-trip needed for logic), reduces server load.
- Cons: depends on browser support, code visible to user (less secure for sensitive logic), can be disabled.

Example:

```
<script>
function validateForm() {
  if (document.getElementById("name").value === "") {
    alert("Name required!");
    return false;
  }
}
</script>
```

6. Server-Side Scripting

- Code that **runs on the web server**, before page sent to client.
- Languages: PHP, Node.js (JavaScript), Python (Django/Flask), Java (JSP/Servlets), ASP.NET, Ruby.
- Used for: database interaction, authentication, business logic, generating dynamic HTML.
- Pros: secure (code hidden from client), full access to server resources/DB.

- Cons: needs server round-trip (slower for interactivity), more server load.

Example (PHP):

```
<?php
$name = $_POST['name'];
echo "Hello, " . $name;
?>
```

Client-Side Scripting	Server-Side Scripting
Runs in browser	Runs on server
Reduces server load	Increases server load
Code visible to user	Code hidden from user
Can't access server DB directly	Full access to DB/server resources
Example: JavaScript	Example: PHP, Node.js, Python

7. Web 1.0, Web 2.0, and Web 3.0

Web 1.0 (~1991–2004) — “Read-Only Web”

- Static pages, one-way information (publisher → reader).
- No user interaction/comments, no accounts.
- Example: early static HTML sites, online brochures.

Web 2.0 (~2004–present) — “Read-Write Web”

- Interactive, user-generated content, social participation.
- AJAX enables dynamic pages without reload.
- Features: blogs, social media, wikis, comments, likes/shares.
- Example: Facebook, YouTube, Wikipedia, Twitter/X.

Web 3.0 — “Read-Write-Execute / Semantic & Decentralized Web”

- Data understood by machines (Semantic Web), AI-driven, personalized.
- Decentralization concepts: blockchain, cryptocurrency, smart contracts.
- Features: intelligent search, AI assistants, decentralized apps (dApps).
- Example: blockchain-based apps, AI-personalized content.

	Web 1.0	Web 2.0	Web 3.0
Nature	Read-only	Read-write	Read-write-execute
Content	Static, by owner	User-generated	AI-curated, decentralized
Interaction	None	High (social)	Intelligent, personalized
Tech	HTML	AJAX, JavaScript frameworks	AI, blockchain, semantic web
Example	Static HTML sites	Facebook, YouTube	dApps, AI assistants

Quick Recap (Exam Points)

- Internet = global network; Intranet = private network; WWW = service on Internet (hypertext documents).
- Static page = fixed content; Dynamic page = generated per request.
- Client requests, Server responds — this is client-server model.
- Tiers: 1-Tier (all on one machine), 2-Tier (client + DB server), Multi-Tier (client + app server + DB server).
- HTTP = stateless protocol; Request = method+URL+headers+body; Response = status+headers+body.
- URL parts: scheme, host, port, path, query, fragment.
- Client-side scripting = browser (JS); Server-side scripting = server (PHP/Node/Python).
- Web 1.0 = read-only, Web 2.0 = read-write (social), Web 3.0 = intelligent/decentralized.